

Chapter #7

ANN

Q. Identify the basic components of ANNs and their properties and capabilities?

Motivation behind ANN:

Human brain vs digital computers:

⇒ The brain process information in a high complex, non linear, and parallel manner, unlike conventional digital computers.

ANNs Basics:

Definition: An ANN is a massive parallel distributed processing model made up of simple processing units.

○ ANNs are abstract computational models inspired by the human brain.

Neurons: it has the ability to learn experimental knowledge encoded through interunit connection strength and can make such knowledge available for use.

○ The human brain has an estimated 10^{11} tiny units called neurons.

○ These neurons are interconnected with an estimated 10^{15} links.

○ Similar to brain ANN composed of artificial neurons and interconnections.

Graphical representations

○ Neurons as nodes and interconnections as edge in the graph.

Other names:

- Neural network
- PDP system
- Connectionist model
- Distributive adaptive system
- Neurocomputers

Other definitions:

⇒ A neural network is a set of connected i/p/o/p units. A weight is associated with each connection, which is adjusted by the network during learning process. So, that, the network is able to predict the correct class labels of i/p tuples.

Structure:

⇒ A network structure consist of a number of nodes connected through directional links.

⇒ Each node represents a processing unit and all are adaptive.

⇒ The link b/w nodes specify the causal relationship b/w connected nodes.

Properties and Capabilities of ANNs

1. Nonlinearity:

⇒ Each artificial neuron can process information in a linear or non-linear way, but the whole ANN function in a highly non-linear manner which is essential for modeling complex real world data.

2. Learning from examples:

⇒ ANNs adjust the weights of their connections based on training data, resulting in tuned parameters that store the necessary knowledge to solve problems.

3. Adaptability:

⇒ ANNs can change their connection weights in response to new data or change in the environment.

⇒ If an ANN is trained for a specific task, it can be retrained for new conditions.

Evidential Response:

⇒ ANNs can provide confidence levels for their decisions, improving performance by rejecting uncertain data.

5. Fault tolerance:

⇒ ANNs are robust.

⇒ ANNs are designed to continue functioning well even if some neurons fail or if the input data are noisy or incomplete.

6. Uniformity of analysis and design:

⇒ The same basic principles, terminology, and methods apply across all applications of ANNs. The consistency simplifies the design and analysis of ANNs for various tasks.

Model of an ANN

⇒ An artificial neuron is an information processing unit that is fundamental to the operation of an ANN.

⇒ Three main components of ANNs are:

1. Connection links (Synapses):

Inputs: The neuron receives multiple inputs denoted as x_i

Weights: Each input has a weight, w_{ki} indicating its importance. Weights can be +ve or -ve.

2. Adder:

Summation:

- o The adder sums up all the ip signals after they have been multiplied by their respective weights. This operation is called linear combination.

3. Activation functions:

Output Limiting:

⇒ The activation function, denoted as f , process the sum from the adder to produce the neuron's output y_k .

⇒ This function limits the output to a certain range, making it manageable and meaningful.

Types:

o Hard limit:

$$y = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$$

o Symmetrical hard limit:

$$y = \begin{cases} 1 & \text{if } net \geq 0 \\ -1 & \text{if } net < 0 \end{cases}$$

o Linear

$$y = net$$

o Saturating linear

$$y = \begin{cases} 1 & \text{if } net > 1 \\ net & \text{if } 0 \leq net \leq 1 \\ 0 & \text{if } net < 0 \end{cases}$$

o Symmetric saturating linear

$$y = \begin{cases} 1 & \text{if } net > 1 \\ net & \text{if } -1 \leq net \leq 1 \\ -1 & \text{if } net < -1 \end{cases}$$

o Log-sigmoid

$$y = \frac{1}{1 + e^{-net}}$$

o Hyperbolic tangent sigmoid

$$y = \frac{e^{net} - e^{-net}}{e^{net} + e^{-net}}$$

Net:

- o The weighted sum of product $x_i w_i$ for $i = 1, \dots, m$.

$$net_k = x_1 w_{k1} + x_2 w_{k2} + x_m w_{km} + b_k$$

where $b_k = w_{k0}$

$$o \quad \sum_{i=0}^m x_i w_{ki}$$

$$o \quad net_k = X \cdot W \quad (\text{Vector notation})$$

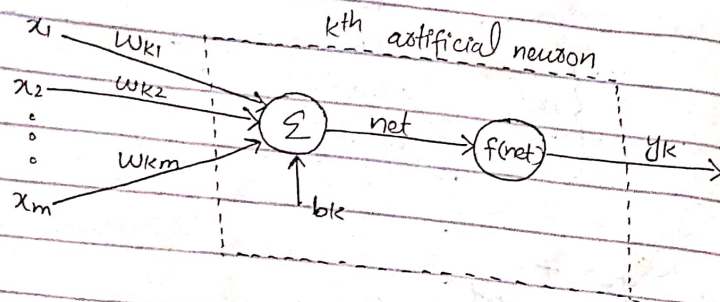
$$X = \{x_0, \dots, x_m\}$$

$$W = \{w_{k0}, \dots, w_{km}\}$$

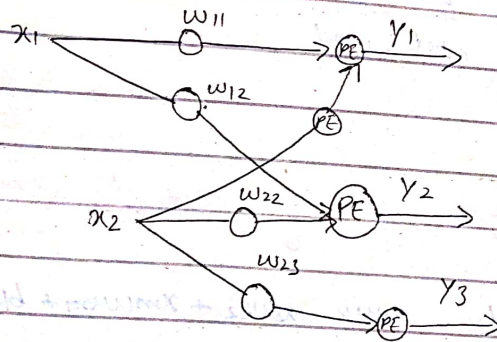
o Outputs

$$y_k = f(\text{net}_k)$$

Block diagram:



Summation function for several neurons:



$$y_1 = w_{11}x_1 + x_2w_{12}$$

$$y_2 = x_1w_{21} + x_2w_{22}$$

$$y_3 = x_1w_{31} + x_2w_{32}$$

Architectures of ANNs

⇒ The architecture of ANN is determined by two main factors:

- o The characteristics of individual nodes.
- o The way nodes are connected within the network.

⇒ Typically, architecture is specified by the:

- o Number of i/p's
- o Number of o/p's
- o Total number of nodes
- o Organization and interconnections of nodes

* Layers *

⇒ ANNs consist of three layers:

1. Input layers:

- o it accepts i/p in several different formats provided by the programmer.

2. Hidden layers:

- o it presents b/w input and o/p layers.
- o it performs all the calculations to find hidden features and patterns.
- o Hidden layers can be more than one in some cases.

5. Output layers

⇒ Contains activation function that choose whether a node should fire or not only those who are fired make it to the o/p layer.

Threshold value:

⇒ A hurdle value for the output of a neuron to trigger the next level of neurons.

⇒ If an output value is smaller than the threshold value, it will not be passed to the next level of neurons.

Types of ANNs

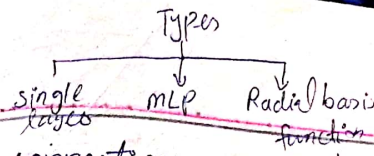
⇒ Common neural network models are:

- Backpropagation
- Feed Forward
- Recurrent network.

Feedforward Network:

Structure:

⇒ Neurons are arranged in layers



and connections are only made forward, from input to output, without loops.

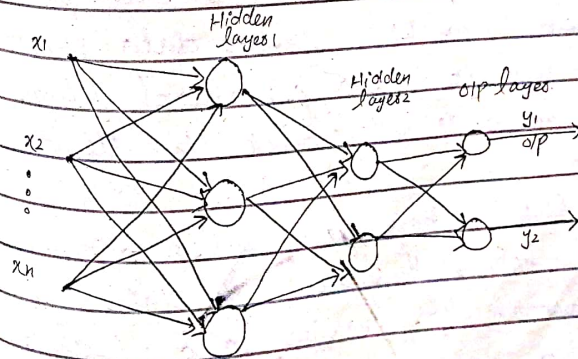
⇒ In this network connections b/w nodes does not form a cycle.

⇒ This is the simplest form of network as information is only processed in one direction.

⇒ While data may pass through multiple hidden nodes, it always move in one direction and never backwards.

⇒ it can be categorised:

- single layer perception
- multi layer perception

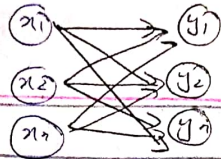


Single-layer

- only two layers
- i/p & o/p
- i/p does not connect to o/p because no computation is performed in this layer.

Usage:

- commonly used for pattern recognition and classification tasks.



2. Recurrent network:

Structure

- Neurons can have connections that loop back to previous layers, allowing for feedback within the network.

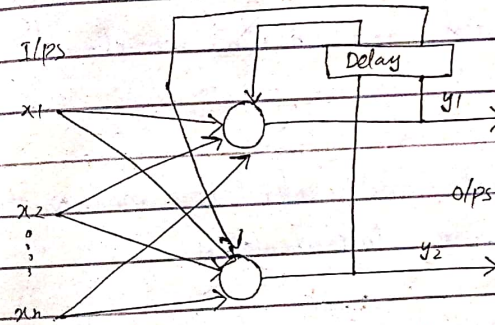
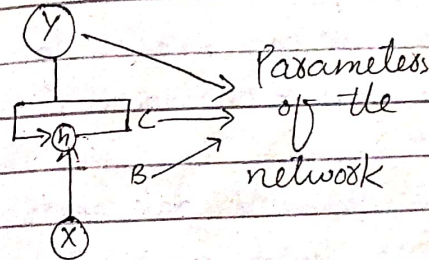
- it is special type that work for time series data or data that involves sequences.

- if we have data ~~point~~ in sequence such that one data point depends upon the previous data point, we need to modify neural network to incorporate dependencies b/w the data points.

- RNNs, have the concept of memory that helps them store the states or information of previous i/p to generate o/p of the sequences.

Types
Competitive network, SOM, ART, etc.

- In RNN output from the previous step are fed as input to the current step.



Usage

- suitable for tasks that involve sequential data or require memory, like time series prediction and natural language processing.

Learning Process in ANN

Objective of learning:

⇒ A major task of ANNs is to learn a model of their environment to achieve specific goals.

⇒ They use real-world data samples for learning, unlike classical systems that rely on predefined mathematical models.

Comparison with classical systems:

★ Classical systems:

- Create and validate a mathematical model before implementation.

★ ANNs:

- Directly use real-world data to learn and process information.

Learning from the environment:

⇒ ANNs improve performance by adjusting connection weights based on data samples.

⇒ Learning is adaptive process influenced by environmental interactions.

Definition of Learning in ANNs:

⇒ Learning involves adapting the neural network's parameters through environmental stimuli.

⇒ The specific method of parameter change determines the type of learning.

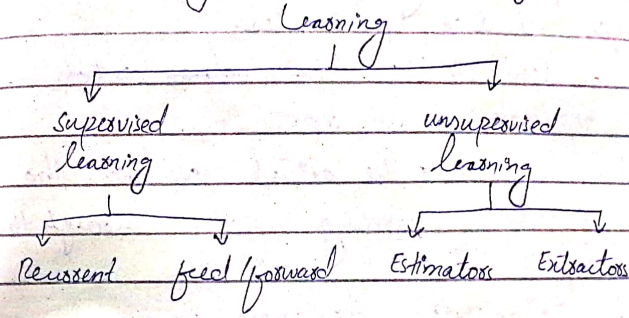
Learning Algorithms

- A set of rules to solve a learning problem.

- Differs in how they adjust the weights.

- Also considers how the ANN architecture is built.

- The training procedure used by ANNs



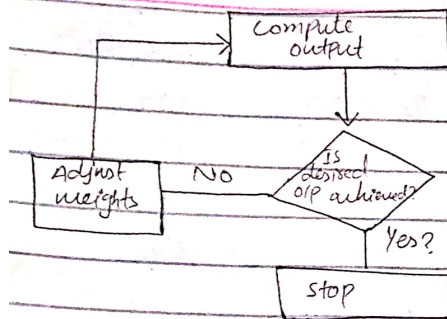
ANN Learning Process:

⇒ The general ANN learning process involves three tasks:

(i) Compute temporary outputs.

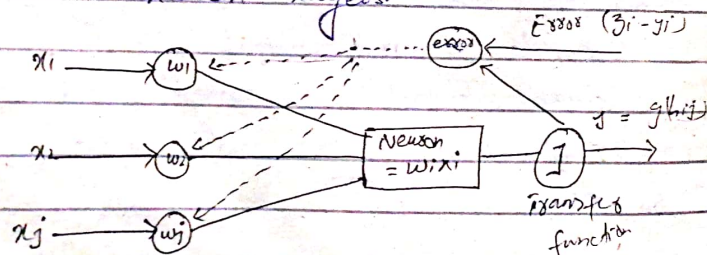
(ii) Compute output with desired target.

(iii) Adjust the weights and repeat the process.



Procedure for a learning Algorithm:

- i- Initialize weights with random values and set other parameters.
- ii- Read in the input values vectors and the desired o/p.
- iii- Compute the actual o/p via the calculations working forward through the layers.
- iv- Compute the errors.
- v- Change the weights by working backward from the o/p layer through the hidden layers.



Learning Tasks using ANN

⇒ The choice of learning algorithm is influenced by the learning task that an ANN is required to perform.

⇒ In different basic learning tasks that apply to the use of different ANNs.

1. Pattern Association:

⇒ Pattern association is a key feature of human memory and cognition.

⇒ Two types:

1. Auto-associations

Def: Storing and retrieving patterns.

- o Type is unsupervised learning.

Process:

Storage phase:

- o Train the network with a set of patterns.

Recall phase:

- o Retrieve a specific pattern when given a partial or noisy version.

2. Heteroassociation:

Def: Pairing different sets of inputs and o/p patterns.

Process:

Storage phase:

- Train the network with paired i/p-o/p patterns.

Recall Phase:

- Retrieve the correct o/p when given an i/p pattern.

Learning type: Supervised Learning

Two main phase in the application of ANN:

1. Storage phase:

- Refers to the training of the network in accordance with given patterns.

2. Recall Phase:

- Refers to retrieval of a memorized pattern in response to the presentation of a noisy or distorted version of a key pattern to the network.

2. Pattern Recognition:

Def:

The process of assigning a received pattern to one of the several predefined classes.

Process:

Training Phase:

- The network is repeatedly presented a set of input patterns along with the category to which each particular pattern belongs.

Testing phase:

- A new, unseen pattern is presented to the network.
- The network identifies the class based on its training.

Graphical representation:

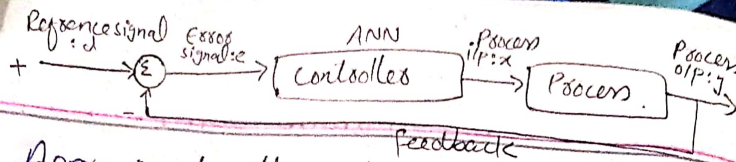
- Patterns are points in multidimensional space called decision space.
- Decision boundaries in the space are determined during training.
- When a new pattern is presented, the network identifies the class based on these boundaries.

Subtypes of PR:

* function approximation:

Def: mapping i/p vectors to o/p vectors when the relationship is unknown.

Objective:



Approximate the unknown function f with an ANN function F such that

$$|F(x_i) - f(x_i)| \leq \epsilon \text{ for all } x_i.$$

Small time numbers

Requirement:

- o A large training set and enough free parameters in the network.

Learning type:

- o Supervised learning

★ Control:

Def:

- o Control is another learning task that can be done by ANN.
- o Control is applied to a process or a critical part in a system, which has to be maintained in a controlled condition.

Process:

Feedback control:

- o Adjust output y to match a reference signal d .
- o it can be trained through

Indirect Learning:

Construct an ANN model off-line using actual i/p - o/p measurements.

Integrate the trained model into the real-time system.

Direct Learning:

Train the ANN Controller online with real-time data.

Adjust free parameters directly from the process feedback.

★ Filtering:

Def:

- o The term filter often refers to a device or algorithm used to extract information about a particular quantity from a set of noisy data.

Tasks:

Filtering:

- o Extract information at discrete time n using data upto and including n .

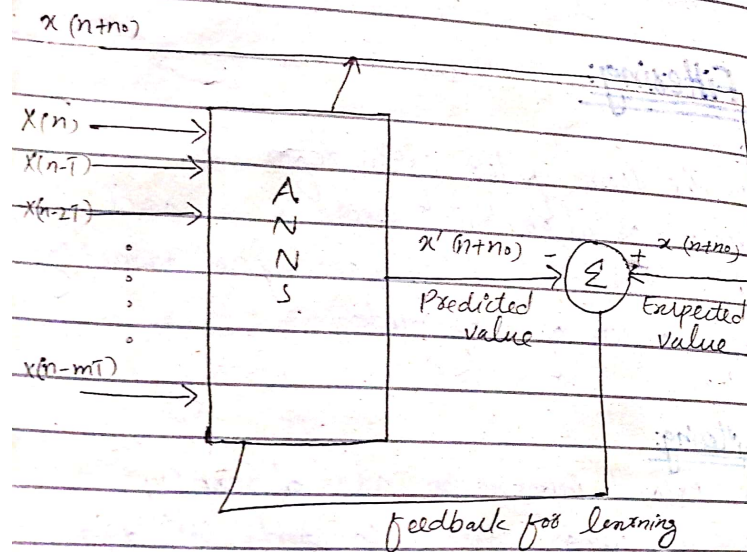
Smoothing:

- o Using data measured later than time n to produce results, allowing a delay in o/p.

Predictions

- Forecast future data values.
- Use data upto n to predict the quantity of interest at $n+n_0$ in the future.
- Smaller prediction errors indicate a better model of the physical process.

Block diagram of ANN Prediction



Multilayer PERCEPTRONS

Deep neural networks

Def: Multilayer feedforward networks are one of the most important and most popular classes of ANN in real-world applications. A fully connected multilayer network.

Characteristics of MLPs:

Nonlinear activation functions

- Neurons use nonlinear activation functions, such as sigmoidal or hyperbolic functions.

Hidden layers:

- Network includes one or more hidden layers of neurons.
- Hidden layers help learn complex, nonlinear task by extracting meaningful features.

High Connectivity

- Neurons in one layer are connected to all neurons in the previous layer.

Structure of MLPs

Input layers:

- Receives input data.

Hidden layers:

- Process data through intermediate layers.

O/p Layer:

- o Produces the final o/p.
- Dataflow:
 - o moves forward from the i/p layer to the o/p layer on a layer-by-layer basis.

Example architecture

Fully connected Networks:

- o Each neuron in one layer connects to every neuron in the previous layer.
- o Data flows from left to right through the network.

Training MLPs:

- o Error Backpropagation algorithm for weight adjustment iteratively in order to minimize loss function.
- Supervised Training:
 - o Uses known i/p-o/p pairs to train the network.

Two Phases of Training

Forward Pass:

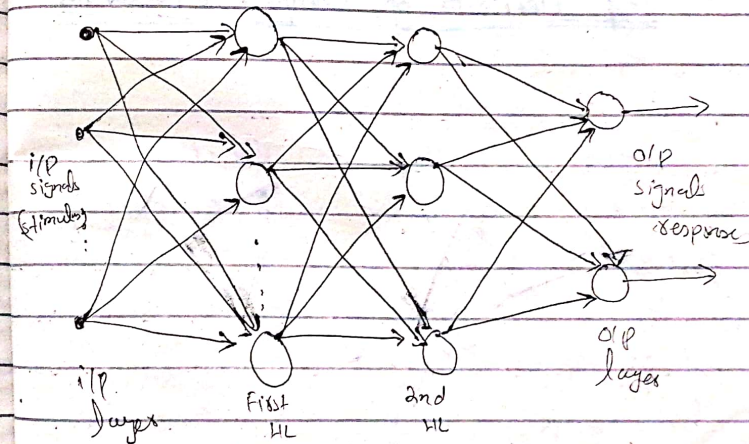
- ⇒ I/p data is applied to the i/p nodes.

data propagates through the network, layer-by-layer.

- o The network produces an o/p response.
- o Synaptic weights remain fixed during this phase.

Backward Pass:

- o Compare the actual o/p with the desired o/p to calculate the error.
- o Error signal is propagated backward through the network.
- o Synaptic weights are adjusted based on the error to improve accuracy.



Competitive Networks & Competitive Learning

⇒ Competitive neural networks belong to a class of recurrent networks, and they are based on algorithms of unsupervised learning such as competitive algorithm. Neurons activations

⇒ In CL the o/p neurons of a neural network compete among themselves to become active (to be "fired"). Only one o/p neuron is active at any one time.

Key Elements of Competitive Networks

1. Set of Neurons

- Neurons with the same structure.
- Initially connected with randomly selected weights.
- Respond differently to the same set of i/p samples.

2. Limit values

- A threshold determined by the strength of each neuron.

Competition mechanisms

Neuron compete to respond to a subset of inputs.
only one neuron is active at a time.

Purpose:

- Used to cluster i/p data with a predefined number of output classes.

Network architecture

Single layer of o/p neurons:

- Fully connected to i/p nodes through synaptic
- may include feedback connections among neurons. Also connected to each other

Winning neuron criteria

- For neuron k to win, its net value net_k for an i/p x must be highest among all neurons.

⇒ if neuron k wins:
o/p $y_k = 1$

⇒ if neuron k loss:
o/p $y_k = 0$

formulae

$$y_k = \begin{cases} 1 & \text{if } net_k > net_j \text{ for all } j \\ 0 & \text{otherwise} \end{cases}$$

Key characteristics of CC

Adaptive classification.

⇒ Efficient at adaptive classification tasks.

Methodological problems:

Learning Rate η :

- Affects speed of learning and stability of weights.

High Learning rate:

- Faster learning but less

stable.

Low learning rates

- Slower learning but more

stable.

Competitive NW & Hamming NW

Relationship:

Both are closely related.

Hamming network structure:

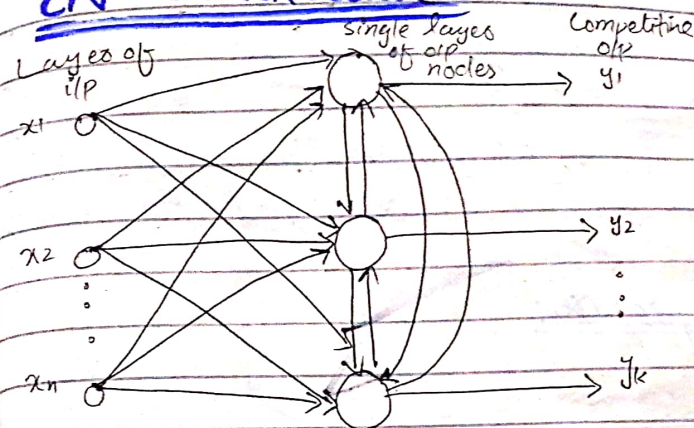
1st layers:

- Standard feedforward layer that correlates the input vectors with preprocessed output vectors.

second layers:

○ Perceptron Competition to find the closest match to the input layers.

CN architecture



have no activation function
self organizing map

SOMs

⇒ used for clustering & dimensionality reduction.
Definition: → self-organizing because it does not need any supervision.

⇒ SOMs, also called Kohonen maps, are a data visualization technique.

⇒ Introduced by Teuvo Kohonen, a professor at the University of Helsinki. > 1982

Purpose: - it learn on their own through unsupervised competitive learning.

- To project high-dimensional input data into a visual representation, usually a 2-D image map. Also called feature map.

Key features:

Visualization:

- Helps visualize and understand high-dimensional data by reducing its dimensions.

Explorations:

- Useful for exploring linear and non-linear relationships in data.

Clustering & Classification:

- Initially used for speech recognition, now applied widely for various clustering and classification tasks.

⇒ SOMs are neural networks that uses unsupervised learning approach and trained its network through ~~loss~~ competitive learning algorithm to map multidimensional data to lower dimensions.

Problem solved:

- Humans cannot visualize high-dimensional data, SOMs transform complex data into a more comprehensible form.

Functionality:

Data grouping:

- Projects data into a lower-dimensional space while preserving topological relationships.

Output map:

- produces a 1-D or 2-D output map that groups similar data items together.

Network structure:

- A single-layer feedforward network with input neurons arranged in 2-D topological grid.

Learning process:

Initialization:

- Neuron weights are initialized randomly or using principal components.

Training:

- Involves sequential and iterative presentation of training data to the SOM.

Two layers $\begin{cases} \rightarrow \text{i/p} - n \text{ units} \\ \rightarrow \text{o/p} - m \text{ units} \end{cases}$

BMU: The neuron whose weight vector is closest to the i/p sample.

Advantages:

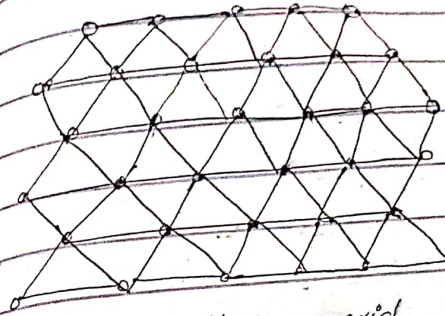
- $\Rightarrow$ Understandable results
- $\Rightarrow$ Simple implementation
- $\Rightarrow$ practical effectiveness

Disadvantages:

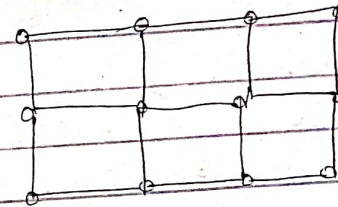
- $\Rightarrow$ Computational cost
- $\Rightarrow$ Sensitivity
- $\Rightarrow$ Data requirements

Applications:

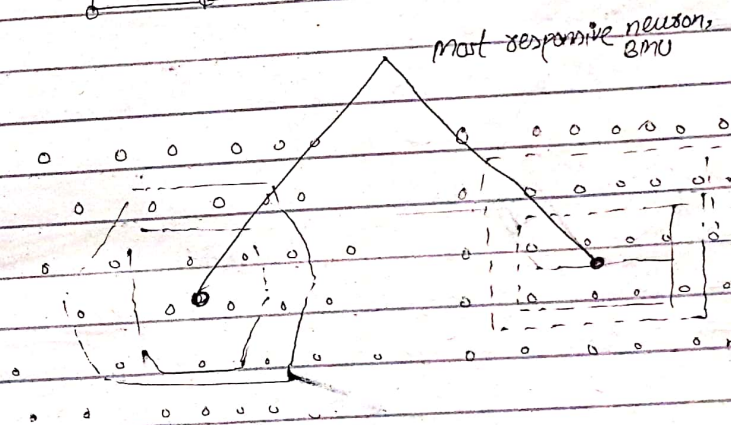
- o Automatic speech recognition
- o Clinical data analysis
- o Genetic information analysis
- o Brain signal analysis
- o Document retrieval
- o Industrial monitoring



Hexagon grid



Rectangle grid



SOMs Algorithm

1) Initialization:

Choose random values for the initial weights w_{ij} .

2) Sampling:

Take a sample training i/p vector from i/p layers.

3) Matching:

Find the winning neuron from the o/p layers that has weight vector closest to the i/p vector. It can be calculated using Euclidean distance for each o/p unit and find the output unit that has minimum Euclidean distance from the i/p vector. for each time

$$D(i) = \sum_{i=1}^n \sum_{j=1}^m (x_i - w_{ij})^2$$

4) New weight calculation

Find the new weights based on i/p vector & winning o/p unit.

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha (x_i - w_{ij}(\text{old}))$$

5) Continuation:

Repeat step 2 & 4 until weight updation is negligible.

Pros of ANN:

- ⇒ Parallel processing capability
- ⇒ Storing data on the entire network
- ⇒ Capability to work with incomplete knowledge
- ⇒ Having fault tolerance

Cons of ANN:

- ⇒ Hardware dependence
- ⇒ unrecognized behavior of the network
- ⇒ Assurance of proper network structure
- ⇒ The duration of network is unknown

Applications of ANN:

- ⇒ NLP
- ⇒ facial analysis
- ⇒ Personal assistant